C++ in Quantitative Finance

#3
Abstract methods, polymorphism and virtual constructors

Paweł Sakowski

Spring 2026

University of Warsaw
Faculty of Economic Sciences

Abstract methods

- By denoting a method as a abstract one (or pure virtual) we inform the compiler that this method must be available, but by now there it has no implementation (definition).
- Responsible for providing implementations of abstract methods are subclasses. They have to define all abstract methods in order to make them usable.
- An example of an abstract method could be the `play()` method in the `Pet` class. Its definition's syntax is:
  `virtual void play()=0 ;`

Polymorphism

- Polymorphism is the ability to use an operator or a method in different ways.
- It gives different meanings or functions to the operators or methods.
- Polymorphism refers to codes, operations, methods or objects that behave differently in different contexts.

The problem for today

- Last time we used a simple Monte Carlo program which used a polymorphic class `PayOff` to determine the pay-off of the vanilla option to be priced.
- However, a natural object we want to model is the option itself.
- At the moment, we have two pieces of data: (1) the expiry, and (2) the pay-off.
- It would be much more natural to have a single piece of data, ie. the option, which encompassed both.
- How to do this?

Possible solution?

One possible solution could be:

1. Copy all the `PayOff` code already written.
2. Add to the base class:
   - an extra data member `Expiry`
   - an extra method `GetExpiry()`

However, this is not the best idea.

- We would like to reuse the same code rather than cut and paste it.
- Moreover, it would be time consuming if we always had to write a new inherited class for any new option type we want to price.

Preferable solution

- Define a `VanillaOption` class which has as data members a `PayOff` object and a `double` to represent expiry.
- We want the `VanillaOption` class to be able to contain an object from an unknown class.
- This could be easily achieved by referring to extra data outside the object using pointers or reference.
- Our solution is therefore to store a reference to a `PayOff` object instead of a `PayOff` object (see project05).
- The main change to the last week is that now we: 1) pass a `PayOff` object and the expiry time into a `VanillaOption` object and 2) we pass the `VanillaOption` object into the Monte Carlo function.

Drawback!

- However, there is a drawback: the `VanillaOption` will not exist as independent object in its own right but will instead always be dependent on the `PayOff` object constructed outside the class.
- Soon or later it will surely bring us troubles.

Virtual construction

- We want the `VanillaOption` to store its own copy of the `PayOff`.
- However, we still don't want the `VanillaOption` to know the type of the `PayOff` object nor anything about any of its inherited classes.
- Our solution last week was to use virtual functions.
- Now, we will define a virtual method of the base class which causes the object to create a copy of itself and return a pointer to this copy (see project06).
- Such a method is called a *virtual copy constructor*.

Other

- Now, the `VanillaOption` class can be happily copied and moved around just like any other class (see project06).
- It uses the code already written for the `PayOff` class so we have no unnecessary duplication.
- However, we had to write special code to handle assignment, construction and destruction.
- Writing it again for any new class would be annoying.
- What we would really like is a `PayOff` class that has the nice polymorphic features that our class does.
- This could be achieved either by using:
  1. *the bridge pattern*
  2. a template class

Exercises I

1. Add to the Project05 two new subclasses: PayOffCustomCall and PayOffCustomPut inheriting after the PayOff base class with the following payoff functions:

   the CALL pays off:

   - 0 if Spot at expiry is smaller or equal than Strike
   - 1 if Spot at expiry is greater than Strike by no more than 5%
   - 2 if Spot at expiry is greater than Strike by more than 5% and less than 10%
   - 3 if Spot at expiry is greater than Strike bo more than 10%

   the PUT pays off:

   - 0 if Spot at expiry is greater or equal than Strike
   - 1 if Spot at expiry is smaller than Strike by no more than 5%
   - 2 if Spot at expiry is smaller than Strike by more than 5% and less than 10%
   - 3 if Spot at expiry is smaller than Strike bo more than 10%

   Find the theoretical price of these two options.

Exercises II

2. Add both subclasses from previous exercise (PayOffCustomCall and PayOffCustomPut) to the Project06.
3. Use the program from Project08 two find prices of two new options:
   - a CALL that pays off $\max(S_T - K_1, 0)$ if $S_T <= K_2$ and $2 * \max(S_T - K_1, 0)$ if $S_T > K_2$
   - a PUT that pays off $\max(K_2 - S_T, 0)$ if $S_T > K_1$ and $2 * \max(K_2 - S_T, 0)$ if $S_T < K_1$

   assuming that for two strikes holds $K_1 > K_2$.

Thank you!

Paweł Sakowski
sakowski@wne.uw.edu.pl